main.py × Terminal | Terminal

```
    img_data = response.content
    img = Image.open(BytesIO(img_data))
    with open(x+'.jpg', 'wb') as handler:
      handler.write(img_data)
    return img

def generate_earth_image(angle):
    prompt = f"Realistic Earth from space at {angle} degrees angle"
    image_url = generate_base_image(prompt)
    image_filename = f"earth_{angle}_degrees"
    image = download_image(image_url, image_filename)
    return image

#angles = range(0, 360, 10)
#earth_images = [generate_earth_image(angle) for angle in angles]

angles = range(0, 360, 10)
image_filenames = [f"earth_{angle}_degrees.jpg" for angle in angles]

resized_earth_images = []
for filename in image_filenames:
    img = Image.open(filename)
    resized_img = img.resize((256, 256), Image.ANTIALIAS)
    resized_earth_images.append(resized_img)

output_gif = "rotating_earth.gif"
resized_earth_images[0].save(
    output_gif,
    save_all=True,
    append_images=resized_earth_images[1:],
    duration=100,
    loop=0
)
```

100% (55:2)

Guide ×

Collapse

Processing the Images

First, let's resize the images so the animation is smaller in size and easier to handle. In the following code, we resize each image in the `earth_images` list to 256x256 pixels:

```
angles = range(0, 360, 10)
image_filenames = [f"earth_{angle}_degrees.jpg" for angle in angles]

resized_earth_images = []
for filename in image_filenames:
    img = Image.open(filename)
    resized_img = img.resize((256, 256), Image.ANTIALIAS)
    resized_earth_images.append(resized_img)
```

TRY IT

```
Command was successfully executed.
```

We create a list of image filenames called `image_filenames` for the Earth images at different angles. We then create an empty list `resized_earth_images` to store the resized images. In the for loop, we open each image using its filename, resize it, and append it to the `resized_earth_images` list.

Creating the GIF Animation

Next, we'll create a GIF animation using the resized images. We can do this using the `save` method of the PIL Image class. We'll set the duration of each frame to 100 milliseconds and loop the animation indefinitely:

```
output_gif = "rotating_earth.gif"
resized_earth_images[0].save(
    output_gif,
    save_all=True,
    append_images=resized_earth_images[1:],
    duration=100,
    loop=0
)
```

TRY IT

```
Command was successfully executed.
```

In this code, we first specify the output GIF filename as `"rotating_earth.gif"`. Then, we call the save method on the first image in the `resized_earth_images` list. We set `save_all=True` to indicate that we want to save multiple frames. The `append_images` parameter is set to the rest of the images in the list, and we specify the duration and loop count using the `duration` and `loop` parameters, respectively.

Now, when you run this code, a GIF animation called "rotating_earth.gif" will be created, showcasing the Earth rotating using the images generated by the DALL-E 2 API.

The images that the AI generated did not all look the same hence our current gif. The different images don't have the same center there fore our gif looks less smooth

0 degrees

270 degrees

350 degrees

Loop PIL

In the following code snippet, what should the loop parameter be set to in order for it to loop indefinitely ?

```
output_gif = "rotating_earth.gif"
resized_earth_images[0].save(
    output_gif,
    save_all=True,
    append_images=resized_earth_images[1:],
    duration=100,
    loop=0
)
```

The loop parameter is used to specify the number of times the animation should loop. In this case, the value of `loop` is set to `0`, which means that the animation will loop indefinitely. If you wanted the animation to loop a specific number of times, you could set the loop parameter to that value (e.g., `loop=3` would make the animation loop three times).

Check It!

Next